SPARQL

Mikel Egaña Aranguren

mikel-egana-aranguren.github.io

mikel.egana@ehu.eus

Universidad del País Vasco
Euskal Herriko Unibertsitatea

BILBOKO INGENIARITZA ESKOLA
ESCUELA DE INGENIERÍA DE BILBAO

SPARQL

https://github.com/mikel-egana-aranguren/ABD

CC BY

SPARQL

SPARQL: SPARQL Protocol and RDF Query Language

Estándar oficial del W3C para:

- Hacer consultas sobre grafos RDF
- Ejecutar esas consultas sobre HTTP(S) contra un SPARQL endpoint

(Incluyendo consultas federadas)

SPARQL

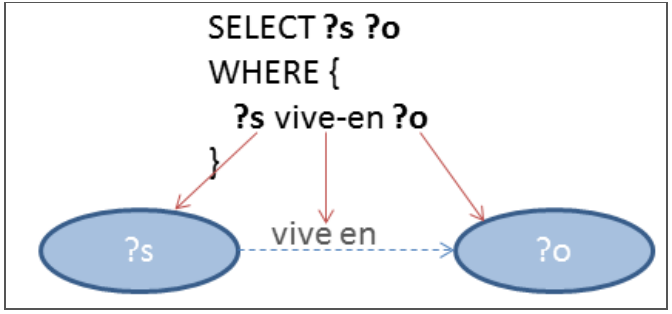
Empresas que piden conocimiento de SPARQL: https://sparql.club/ (Datos de indeed.com y monster.com)

Libros: https://www.learningsparql.com/

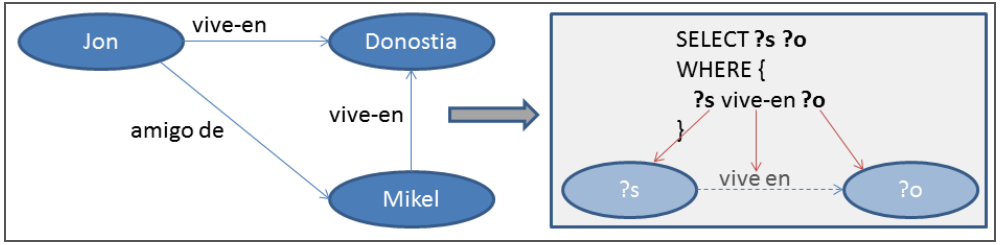
SPARQL

```
SELECT ?s ?o

WHERE {

    ?s vive_en ?o

}
```

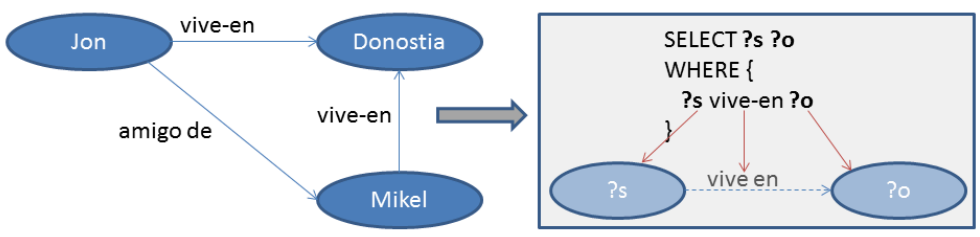
SPARQL



SPARQL


Jon
vive-en
Donostia
amigo de
vive-en
Mikel
SELECT ?s ?o
WHERE {
?s vive-en ?o
}
?s
vive en
?o

SPARQL



?s	?o
Jon	Donostia
Mikel	Donostia

Estructura de la consulta

(Subir museoak.rdf a GraphDB) (1.rq)

```
# Prefixes
PREFIX gip_prop: <http://gipuzkoa.eus/prop/>
PREFIX gip: <http://gipuzkoa.eus/resource/>

# Variables que queremos recibir
SELECT ?sujeto ?clase

# Patrón del grafo que queremos extraer del grafo mayor
WHERE {
  ?sujeto gip_prop:bizilekua gip:donostia .
  ?sujeto rdf:type ?clase
}
```

Estructura de la consulta

(2.rq)

```
# Prefixes
PREFIX gip_prop: <http://gipuzkoa.eus/prop/>
PREFIX gip: <http://gipuzkoa.eus/resource/>

# Queremos recibir todas las variables
SELECT *

# Patrón del grafo que queremos extraer del grafo mayor
WHERE {
  ?sujeto gip_prop:bizilekua gip:donostia .
  ?sujeto rdf:type ?clase
}
```

Optional

(3.rq)

```
PREFIX gip_prop: <http://gipuzkoa.eus/prop/>
PREFIX gip: <http://gipuzkoa.eus/resource/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT ?langile ?bizilekua

WHERE {
  # Tiene que ser una persona
  ?langile rdf:type foaf:person .

  # Tiene que vivir en algun sitio
  ?langile gip_prop:bizilekua ?bizilekua
}
```

Optional

(4.rq)

```
PREFIX gip_prop: <http://gipuzkoa.eus/prop/>
PREFIX gip: <http://gipuzkoa.eus/resource/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT ?langile ?bizilekua

WHERE {
  # Tiene que ser una persona
  ?langile rdf:type foaf:person .
  # Puede vivir en un sitio o no
  OPTIONAL {
     ?langile gip_prop:bizilekua ?bizilekua
  }
}
```

Union

(5.rq)

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?entitatea ?izena

WHERE {
  {?entitatea rdfs:label ?izena }
  UNION
  {?entitatea foaf:name ?izena }
}
```

Ordenar resultados

(6.rq)

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?pertsona ?izena

WHERE {
  ?pertsona foaf:name ?izena
}


ORDER BY DESC (?izena) # Puede ser DESC o ASC
```

Limitar resultados

(7.rq)

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?pertsona ?izena

WHERE {
  ?pertsona foaf:name ?izena
}
ORDER BY DESC (?izena)
LIMIT 3
```

Paginación

(8.rq)

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?pertsona ?izena

WHERE {
  ?pertsona foaf:name ?izena
}
ORDER BY DESC (?izena)
LIMIT 3
OFFSET 3
```

Paginación

(9.rq)

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?pertsona ?izena

WHERE {
  ?pertsona foaf:name ?izena
}
ORDER BY DESC (?izena)
LIMIT 3
OFFSET 6
```

Filtrar resultados

(10.rq)

```
PREFIX gip_prop: <http://gipuzkoa.eus/prop/>

SELECT ?museoa ?langile_kop

WHERE {
    ?museoa gip_prop:kopurua ?langile_kop .
    FILTER (?langile_kop > "800"^^xsd:int)
}
```

Filtrar resultados

(11.rq)

```
SELECT ?langile ?izena

WHERE {
    ?langile foaf:name ?izena .
    FILTER regex(?izena,'Mi.*')
}
```

Filtrar resultados

Lógica: !, &&, ||

Cálculos: +, -, *, /

Comparaciones: =, !=, >,<

Filtrar resultados

Tests SPARQL: isURI, isBlank, isLiteral, bound

Acceder a datos: str, lang, datatype

Más: sameTerm, langMatches, regex, …

Evitar duplicados

(12.rq)

```
SELECT ?lantokia
WHERE {
  ?person rdf:type foaf:person .
  ?person <http://vocab.data.gov/def/drm#worksFor> ?lantokia
}



SELECT DISTINCT ?lantokia
WHERE {
  ?person rdf:type foaf:person .
  ?person <http://vocab.data.gov/def/drm#worksFor> ?lantokia
}
```

Ask

(13.rq)

```
PREFIX gov:<http://vocab.data.gov/def/drm#>
PREFIX gip:<http://gipuzkoa.eus/resource/>

ASK
WHERE {
  ?person gov:worksFor gip:gugenheim
}
```

Describe

```
DESCRIBE <http://gipuzkoa.eus/resource/mikel-aranguren>
```

CONSTRUCT

(14.rq)

```
PREFIX gip_prop:<http://gipuzkoa.eus/prop/>
PREFIX gip:<http://gipuzkoa.eus/resource/>
PREFIX gov:<http://vocab.data.gov/def/drm#>

CONSTRUCT {
   ?langile rdf:type gip:gugenheim_langilea
}
WHERE {
   ?langile gov:worksFor gip:gugenheim
}
```

Delete data

```
DELETE DATA {
            <http://gipuzkoa.eus/resource/aitor-labajo>
                rdf:type foaf:person
          }
```

DESCRIBE <http://gipuzkoa.eus/resource/aitor-labajo>

Delete

```
DELETE {?person rdf:type foaf:person}
WHERE {?person foaf:name ?name}
SELECT * WHERE {
        ?person rdf:type foaf:person
    }
```

INSERT DATA

```
PREFIX gip:<http://gipuzkoa.eus/resource/>
INSERT DATA {
    gip:aitor-labajo rdf:type gip:hiritar
}
DESCRIBE gip:aitor-labajo
```

INSERT

(15.rq)

```
PREFIX gip:<http://gipuzkoa.eus/resource/>

INSERT {
    gip:jon-alfaro rdf:type ?type .
}
WHERE {
    gip:aitor-labajo rdf:type ?type .
}
```

DESCRIBE <http://gipuzkoa.eus/resource/jon-alfaro>

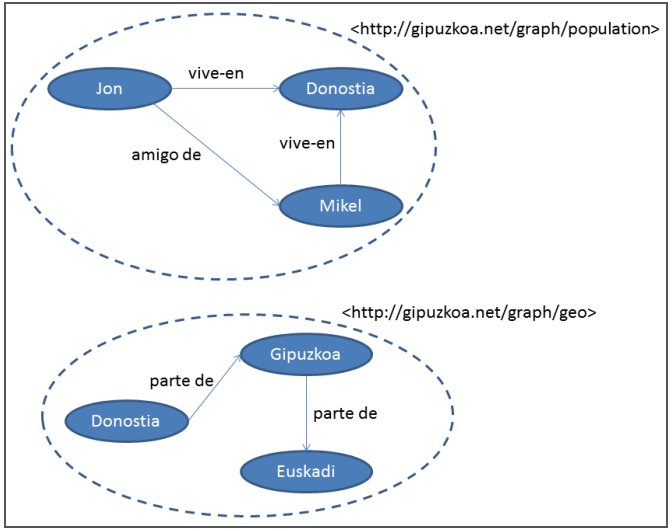
GRAFOS

Grafo: conjunto de triples

El conjunto entero se identifica con una URI (diferente de la de los datos)

Todas las Triple Stores tienen un Default Graph

GRAFOS

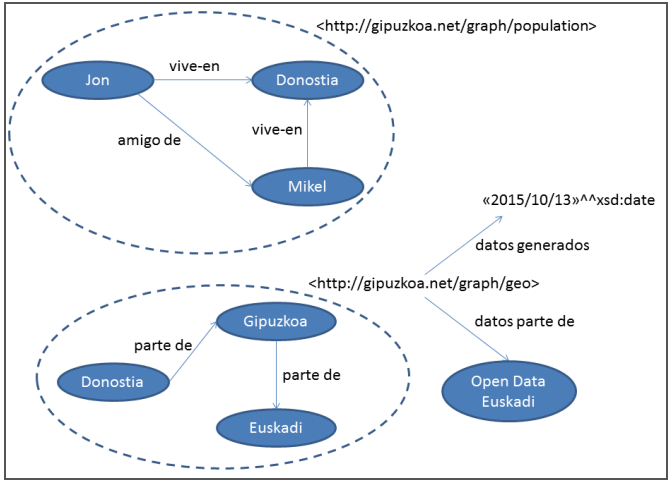

<http://gipuzkoa.net/graph/population>
Jon
vive-en
Donostia
amigo de
vive-en
Mikel
<http://gipuzkoa.net/graph/geo>
Gipuzkoa
parte de
Donostia
parte de
Euskadi

GRAFOS

Los grafos son muy útiles para añadir metadatos (entre otras cosas): ej. procedencia, autoría, fecha de generación

PROV-O, DCAT, …

GRAFOS


<http://gipuzkoa.net/graph/population>
Jon
vive-en
Donostia
amigo de
vive-en
Mikel
«2015/10/13»^^xsd:date
datos generados
<http://gipuzkoa.net/graph/geo>
datos parte de
Gipuzkoa
parte de
Donostia
parte de
Euskadi
Open Data
Euskadi

GRAFOS

(17.rq)

```
PREFIX dc: <http://purl.org/dc/elements/1.1/>
PREFIX ns: <http://example.org/ns#>

INSERT DATA{
  GRAPH <http://example/bookStore> {
    <http://example/book1>  ns:price  42
  }
}
```

GRAFOS

(18.rq)

```
SELECT *
FROM <http://example/bookStore>
WHERE {?s ?p ?o}


SELECT *
FROM <http://example/bookStore>
WHERE {?s ?p ?o}
```

Consultas federadas

(16.rq)

```
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX gip: <http://gipuzkoa.eus/resource/>

SELECT ?pop
WHERE {
    gip:bilbo owl:sameAs ?town
    SERVICE <https://query.wikidata.org/sparql> {
        ?town <http://www.wikidata.org/prop/direct/P1082> ?pop .
    }
}
```

(bilbo_wikidata.ttl)

SPARQL Aggregate Functions

Funciones de agregación para calcular valores sobre grupos:

COUNT: contar elementos

SUM: suma de valores

AVG: promedio

MIN / MAX: mínimo y máximo

GROUP_CONCAT: concatenar valores

COUNT - Ejemplo

```
PREFIX foaf: <http://xmlns.com/foaf/0.1/>


SELECT (COUNT(?person) AS ?total)
WHERE {
    ?person rdf:type foaf:Person
}
```

Cuenta el número total de personas en el grafo

GROUP BY

Agrupar resultados por valores de una variable

```
SELECT ?ciudad (COUNT(?museo) AS ?numMuseos)
WHERE {
    ?museo rdf:type ex:Museo ;
           ex:ubicadoEn ?ciudad
}
GROUP BY ?ciudad
ORDER BY DESC(?numMuseos)
```

HAVING

Filtrar grupos después de la agregación

```
SELECT ?ciudad (COUNT(?museo) AS ?numMuseos)
WHERE {
    ?museo rdf:type ex:Museo ;
           ex:ubicadoEn ?ciudad
}
GROUP BY ?ciudad
HAVING (COUNT(?museo) > 5)
```

BIND

Asignar valores a variables dentro de la consulta

```
SELECT ?person ?fullName
WHERE {
    ?person foaf:firstName ?first ;
            foaf:lastName ?last .
    BIND(CONCAT(?first, " ", ?last) AS ?fullName)
}
```

Funciones de cadenas

CONCAT: concatenar strings

STRLEN: longitud de string

SUBSTR: subcadena

UCASE / LCASE: mayúsculas / minúsculas

STRSTARTS / STRENDS / CONTAINS: búsqueda en strings

REPLACE: reemplazar texto

REGEX - Ejemplo

```
SELECT ?museo ?nombre
WHERE {
    ?museo rdf:type ex:Museo ;
           foaf:name ?nombre .
    FILTER REGEX(?nombre, "arte", "i")
}
```

Encuentra museos con "arte" en el nombre (case-insensitive)

Funciones numéricas

ABS: valor absoluto

ROUND / CEIL / FLOOR: redondeo

RAND: número aleatorio

Operadores: +, -, *, /

```
SELECT ?price (?price * 1.21 AS ?priceWithVAT)
WHERE {
```

Funciones de fecha/hora

NOW(): fecha y hora actual

YEAR / MONTH / DAY: extraer componentes

HOURS / MINUTES / SECONDS: hora específica

```
SELECT ?event ?date
WHERE {
    ?event ex:date ?date .
    FILTER (YEAR(?date) = 2025)
```

VALUES

Proporcionar valores inline para variables

```
SELECT ?city ?population
WHERE {
    VALUES ?city {
        <http://dbpedia.org/resource/Bilbao>
        <http://dbpedia.org/resource/Donostia>
    }
    ?city dbo:population ?population
```

MINUS

Excluir resultados que coinciden con un patrón

```
SELECT ?person
WHERE {
    ?person rdf:type foaf:Person .
    MINUS {
        ?person foaf:age ?age .
        FILTER (?age < 18)
    }
```

NOT EXISTS

Filtrar basándose en la no existencia de un patrón

```
SELECT ?person
WHERE {
    ?person rdf:type foaf:Person .
    FILTER NOT EXISTS {
        ?person foaf:email ?email
    }
}
```

EXISTS vs MINUS

EXISTS/NOT EXISTS: filtro, no elimina bindings

MINUS: elimina soluciones completas

NOT EXISTS es más eficiente para validación

MINUS es más útil para exclusión de patrones complejos

Property Paths

Expresiones de caminos entre nodos:

^: inverso (ex:parent^ encuentra hijos)

: cero o más (ex:knows para red de conocidos)

+: uno o más (ex:subClassOf+ para jerarquía)

?: opcional (cero o uno)

|: alternativa (ex:email|ex:phone)

Property Paths - Ejemplo

```
SELECT ?person ?ancestor
WHERE {
    ?person foaf:name "Mikel" .
    ?person ex:parent+ ?ancestor
}
```

Encuentra todos los ancestros (padres, abuelos, etc.)

Negated Property Sets

Excluir propiedades específicas en property paths

```
SELECT ?subject ?object
WHERE {
    ?subject !rdf:type ?object
}
```

Cualquier relación excepto rdf:type

SPARQL UPDATE

Operaciones de modificación de grafos:

INSERT DATA: insertar triples directamente

DELETE DATA: eliminar triples específicos

DELETE/INSERT: basado en patrones WHERE

LOAD: cargar RDF desde URI

CLEAR: eliminar todo un grafo

DELETE/INSERT combinado

```
DELETE { ?person ex:email ?oldEmail }
INSERT { ?person ex:email ?newEmail }
WHERE {
    ?person foaf:name "Mikel" ;
            ex:email ?oldEmail .
    BIND("mikel@new.com" AS ?newEmail)
}
```

Actualizar email de manera transaccional

COPY / MOVE / ADD

COPY: copiar un grafo a otro (sobreescribe destino)

MOVE: mover grafo (copia + elimina origen)

ADD: añadir triples al destino (sin sobreescribir)

```
COPY GRAPH <http://example.org/graph1>
TO GRAPH <http://example.org/graph2>
```

SPARQL Endpoints públicos

Wikidata Query Service: millones de entidades

DBpedia SPARQL: Wikipedia estructurada

UniProt SPARQL: proteínas y genes

EBI RDF Platform: bioinformática

datos.gob.es: open data España

SPARQL 1.1 Protocol

HTTP GET con parámetro query:

`GET /sparql?query=SELECT...`

HTTP POST (para queries largas):

`POST /sparql` con body URL-encoded

Content negotiation: Accept header

Formatos: application/sparql-results+json, text/csv, application/rdf+xml,

SPARQL desde código

Python: RDFLib, SPARQLWrapper

Java: Apache Jena, RDF4J

JavaScript: SPARQL.js

R: SPARQL package

SPARQLWrapper - Ejemplo Python

```
from SPARQLWrapper import SPARQLWrapper, JSON


sparql = SPARQLWrapper("https://query.wikidata.org/sparql")
sparql.setQuery("""
    SELECT ?city ?cityLabel WHERE {
        ?city wdt:P31 wd:Q515 .
        SERVICE wikibase:label {
            bd:serviceParam wikibase:language "en"
```

SPARQL Performance: Buenas prácticas

Usar LIMIT para pruebas y desarrollo

Poner filtros más restrictivos primero

Evitar OPTIONAL innecesarios (costosos)

Usar FILTER lo más pronto posible en WHERE

Aprovechar índices: triple patterns específicos

Evitar regex complejos si es posible

SPARQL y razonamiento

Razonamiento en tiempo de carga: materialización

Razonamiento en tiempo de consulta: inferencia dinámica

GraphDB: soporta ambos modos

SPARQL puede consultar triples inferidos por OWL/RDFS

Entailment regimes: RDF, RDFS, OWL Direct, OWL RDF-Based

SPARQL 1.1 Service Description

Metadatos sobre el endpoint SPARQL

Descubrir capacidades del servidor

```
SELECT * WHERE {
    <http://example.org/sparql> ?p ?o
}
```

Información: datasets, grafos, extensiones, límites

SPARQL GeoSPARQL

Extensión para consultas geoespaciales

Funciones: geof:distance, geof:within, geof:intersects

```
SELECT ?place ?location WHERE {
    ?place geo:hasGeometry/geo:asWKT ?location .
    FILTER(geof:distance(?location, "POINT(2.17 41.39)"^^geo:w
}
```

Encuentra lugares a menos de 10km de Barcelona

SPARQL Full-Text Search

Extensión para búsqueda de texto (no estándar)

GraphDB: `PREFIX luc:`

`<http://www.ontotext.com/connectors/lucene#>`

```
SELECT ?person ?name ?score WHERE {
    ?search a luc:index ;
            luc:query "Mikel*" ;
            luc:entities ?person ;
```

SPARQL Debugging

Construir query incrementalmente

Usar SELECT * para ver todas las variables

Comentar cláusulas (# comentario)

LIMIT pequeño durante desarrollo

Herramientas: YASGUI (editor online)

Explicar plan de ejecución (vendor-specific)

SPARQL vs SQL

Similitudes: SELECT, WHERE, FILTER (WHERE), ORDER BY, LIMIT

Diferencias:

- SPARQL: patrón de grafos; SQL: tablas relacionales
- SPARQL: datos distribuidos, federación; SQL: local
- SPARQL: esquema flexible; SQL: esquema rígido
- SPARQL: URIs globales; SQL: IDs locales

Cómo lograr un UPSERT en triplestores

1. Carga el delta en un grafo de “staging”.
2. Ejecuta SPARQL Update: DELETE de valores antiguos + INSERT de los nuevos.

Patrón común: GraphDB, Fuseki/Jena, Virtuoso, Stardog, etc.

Ejemplo: DELETE/INSERT dirigido

```
PREFIX ex: # 1) Delta en grafo de staging con el nuevo color # GRAPH <staging>: ex:mi_mascota
ex:color "negro" # 2) Update dirigido sobre el grafo objetivo WITH <mascotas> DELETE { ?s
ex:color ?old } INSERT { ?s ex:color "negro" } WHERE { GRAPH <staging> { ?s ex:color "negro" }
OPTIONAL { ?s ex:color ?old } FILTER( !bound(?old) || ?old != "negro" ) }
```

Idea: borrar valores previos de los sujetos afectados e insertar el nuevo.

Como detectar que hay que actualizar? En SQL se asume que cada fila tiene un solo valor para cada columna, pero en RDF un sujeto puede tener múltiples valores para una propiedad OWL functional property para detectar SHACL para detectar RDF-star para marcar el triple como de "valor unico"

Recursos para aprender SPARQL

SPARQL 1.1 Specification

Learning SPARQL (libro)

Wikidata SPARQL Tutorial

Wikidata Query Service (con ejemplos)

Cambridge Semantics SPARQL Course